

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ
Anul 2013 - Informatică

Cuprins

I. Rezolvare Definitivat Varianta 3 (18 iulie 2013)	2
SUBIECTUL I (30 de puncte)	2
1. Metoda Greedy	2
2. Structura sistemelor de calcul: Unitatea Centrală de Procesare (UCP)	2
3. Programare: Perechi de numere prietene (amicabile)	2
SUBIECTUL al II-lea (30 de puncte)	4
1. Mijloc de învățământ pentru conținutul A: Subprograme recursive	4
a) Caracteristici și avantaje	4
b) Exemplu de valorificare didactică	4
2. Test practic pentru conținutul A: Subprograme recursive	5
SUBIECTUL al III-lea (30 de puncte)	5
Finalitățile educației	5

I. Rezolvare Definitivat Varianta 3 (18 iulie 2013)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Metoda Greedy

- **Principiu:** Metoda Greedy rezolvă probleme de optimizare în care soluția se construiește pas cu pas. La fiecare pas, se alege elementul cel mai optim local (care pare cel mai bun în acel moment), sperând că această succesiune de alegeri locale va conduce la un optim global. Nu se revine niciodată la alegerile deja făcute.
- **Oportunitate:** O problemă are proprietatea de alegere Greedy dacă un optim global poate fi obținut prin alegeri locale optime.
- **Problemă (Problema Rucsacului - varianta fracționară):**

► C++:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
struct Obiect {
    double greutate, valoare, raport;
};
bool cmp(Obiect a, Obiect b) { return a.raport > b.raport; }
double greedyKnapsack(vector<Obiect> &obiecte, double capacitate) {
    sort(obiecte.begin(), obiecte.end(), cmp);
    double valoare_totala = 0.0;
    for (auto &obj : obiecte) {
        if (capacitate >= obj.greutate) {
            capacitate -= obj.greutate;
            valoare_totala += obj.valoare;
        } else {
            valoare_totala += capacitate * obj.raport;
            break;
        }
    }
    return valoare_totala;
}
```

—

2. Structura sistemelor de calcul: Unitatea Centrală de Procesare (UCP)

- **Componente:** Unitatea Aritmetico-Logică (UAL), Unitatea de Control (UC), Regiștrii de memorie interni.
- **Funcții:**
 1. **Faza de decodificare:** Identificarea instrucțiunii ce urmează a fi executată.
 2. **Faza de execuție:** Efectuarea operațiilor aritmetice sau logice corespunzătoare.
 3. Gestiunea fluxului de date între memoria principală (RAM) și dispozitivele periferice.

—

3. Programare: Perechi de numere prietene (amicabile)

Soluție C++:

```
#include <iostream>
#include <fstream>
```

```
using namespace std;

void sumap(long long n, long long &s) {
    s = 1;
    for (long long d = 2; d * d <= n; ++d) {
        if (n % d == 0) {
            s += d;
            if (d * d != n) {
                s += n / d;
            }
        }
    }
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    ofstream fout("DATE.TXT");
    int count = 0;
    long long x = 2;
    while (count < n) {
        long long y;
        sumap(x, y);
        if (y > x) {
            long long x2;
            sumap(y, x2);
            if (x2 == x) {
                fout << x << " " << y << "\n";
                count++;
            }
        }
        x++;
    }
    fout.close();
    return 0;
}
```

Soluție Pascal:

```
program NumerePrietenie;
var
    n, count: integer;
    x, y, x2: int64;
    fout: text;

procedure sumap(n_val: int64; var s: int64);
var
    d: int64;
begin
    s := 1;
    d := 2;
    while d * d <= n_val do
    begin
        if n_val mod d = 0 then
        begin
            s := s + d;

```

```

        if d * d <> n_val then
            s := s + (n_val div d);
        end;
        d := d + 1;
    end;
end;

begin
    readln(n);
    assign(fout, 'DATE.TXT');
    rewrite(fout);
    count := 0;
    x := 2;
    while count < n do
        begin
            sumap(x, y);
            if y > x then
                begin
                    sumap(y, x2);
                    if x2 = x then
                        begin
                            writeln(fout, x, ' ', y);
                            count := count + 1;
                        end;
                    end;
                x := x + 1;
            end;
        close(fout);
    end.

```

—

SUBIECTUL al II-lea (30 de puncte)

1. Mijloc de învățământ pentru conținutul A: Subprograme recursive

Mijloc ales: mediu de programare cu depanator pas cu pas și vizualizare a stivei apelurilor.

a) Caracteristici și avantaje

- Permite rularea instrucțiunilor pas cu pas, astfel încât elevii pot observa intrarea în subprogram, valorile parametrilor și revenirea din apel.
- Vizualizează stiva apelurilor, ceea ce face concretă ideea de apel recursiv și condiție de oprire.

Avantaje față de o prezentare exclusiv verbală:

- Elevii văd efectiv cum se acumulează apelurile recursive și unde se întorc rezultatele.
- Erorile tipice, precum lipsa condiției de oprire sau modificarea greșită a parametrului, pot fi observate imediat prin rulare.

b) Exemplu de valorificare didactică

- **Activitate de învățare:** calculul recursiv al factorialului.
- **Metodă:** demonstrația combinată cu conversația euristică.
- **Formă de organizare:** frontal pentru prima rulare, apoi individual la calculator.

Scenariu:

- Profesorul pornește de la relația matematică $n! = n * (n - 1)!$ și cere elevilor să identifice valoarea de oprire.

- Profesorul scrie subprogramul `fact(n)` și rulează pas cu pas pentru $n=4$, oprindu-se la fiecare apel pentru a evidenția parametrul curent.
- Elevii notează succesiunea apelurilor `fact(4)`, `fact(3)`, `fact(2)`, `fact(1)` și revenirea valorilor 1, 2, 6, 24.
- Elevii modifică programul pentru suma primelor n numere naturale, aplicând aceeași schemă recursivă.

Exemplu C++:

```
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

2. Test practic pentru conținutul A: Subprograme recursive

Testul urmărește competențele de înțelegere, urmărire și implementare a subprogramelor recursive.

Item	Enunț	Răspuns așteptat / etape
1	Precizați condiția de oprire pentru un subprogram recursiv care calculează suma cifrelor unui număr natural.	$n = 0$, când nu mai există cifre de prelucrat.
2	Urmăriți apelurile pentru $f(4)$, unde $f(n)=n+f(n-1)$, $f(0)=0$.	$f(4)=4+3+2+1+0=10$.
3	Scrieți un subprogram recursiv care calculează suma primelor n numere naturale.	Antet corect, caz de bază $n=0$, apel $n + suma(n-1)$.
4	Transformați recursivitatea de la itemul 3 într-o variantă iterativă.	Bucă de la 1 la n , acumulare într-o variabilă s .
5	Explicați de ce un subprogram recursiv fără caz de bază este incorect.	Produce apeluri infinite până la epuizarea stivei; programul nu se termină normal.

—

SUBIECTUL al III-lea (30 de puncte)

Finalitățile educației

Finalitățile educației reprezintă orientările valorice și proiective ale activității educaționale. Ele indică direcția formării personalității și răspund la întrebarea: ce tip de om și ce tip de competențe urmărește sistemul educațional să formeze?

Clasificare:

- finalități macrostructurale: idealul educațional;
- finalități intermediare: scopurile educației;
- finalități operaționale: obiectivele educaționale formulate pentru lecții sau secvențe concrete.

Idealul educațional exprimă modelul de personalitate dorit de societate: o persoană autonomă, responsabilă, capabilă de integrare socială, profesională și culturală. Are caracter general, orientativ și stabil pe termen lung.

Scopurile educaționale concretizează idealul pe trepte de școlaritate, discipline sau arii curriculare. De exemplu, la informatică, un scop este formarea gândirii algoritmice și a capacității de utilizare responsabilă a tehnologiei.

Obiectivele educaționale sunt enunțuri concrete privind achizițiile elevilor. Ele pot fi cognitive, afective sau psihomotorii. Un obiectiv bine formulat precizează comportamentul observabil, condițiile de realizare și criteriul de performanță.

Proceduri de operaționalizare: Pentru a transforma un scop general într-un obiectiv operațional se folosesc verbe observabile: „definește”, „identifică”, „implementează”, „compară”, „argumentează”. Exemplu: „La finalul lecției, elevul va implementa în C++ un subprogram recursiv pentru calculul factorialului, folosind corect condiția de oprire și apelul recursiv.”